Verification Plan
GCD Module (UPC MIRI – PV)

Dan Hagen

June 20, 2026

Contents

1 Design Description

The DUT is a parameterized N-bit Greatest Common Divisor (GCD) module that computes $\gcd(A, B)$ for two unsigned operands A and B using the subtractive Euclidean algorithm. Operands and result are `WIDTH` bits wide. The module exchanges data through two independent ready/valid handshake channels: an input channel (`in_valid`/`in_ready`) that accepts operand pairs and an output channel (`out_valid`/`out_ready`) that delivers the result. Internally it follows a three-state FSM (`IDLE` → `RUN` → `DONE` → `IDLE`) with synchronous active-low reset. Latency is one cycle for early-exit operands ($A = 0$, $B = 0$, or $A = B$) and $1 + N$ cycles for the general case, where $N \geq 1$ is the number of subtractive iterations.

Parameters

Parameter	Default	Description
`WIDTH`	16	Width of `a_in`, `b_in`, and `gcd_out`. Must be ≥ 1.

Top-Level Signal Interface

Signal	Direction	Width	Description
`clk`	Input	1	Clock signal
`rst_n`	Input	1	Active-low synchronous reset
`in_valid`	Input	1	Producer asserts that `a_in` and `b_in` are valid
`in_ready`	Output	1	Module is ready to accept an input transaction
`a_in`	Input	WIDTH	First operand
`b_in`	Input	WIDTH	Second operand
`out_valid`	Output	1	Module asserts that `gcd_out` is a valid result
`out_ready`	Input	1	Consumer is ready to accept the output transaction
`gcd_out`	Output	WIDTH	GCD result

A transaction is captured on the rising edge where `in_valid` and `in_ready` are both high; `a_in`/`b_in` must remain stable while `in_valid = 1` and `in_ready = 0`. The output transaction completes on the rising edge where `out_valid` and `out_ready` are both high; `gcd_out` must be driven from a dedicated result register and remain stable for the entire `out_valid` period.

FSM States

State	in_ready	out_valid	Description
IDLE	1	0	Waiting for input. Ready to accept a new transaction.
RUN	0	0	Iterative subtraction in progress. No new input accepted.
DONE	0	1	Result ready. Waiting for the consumer to accept it.

Reset Behavior

Reset is active-low and synchronous: `rst_n` is sampled on the rising edge of `clk` and must not appear in the `always_ff` sensitivity list. While `rst_n = 0`, on every rising edge the FSM is forced to `IDLE`, all internal registers are cleared, and the module presents `in_ready = 1`, `out_valid = 0`, `gcd_out = 0`. On the first rising edge after `rst_n` is released the module is in `IDLE` and ready to accept input immediately.

2 Traceability Matrix

Each requirement extracted from the specification is traced below to the design feature it characterises, the test that exercises it, and the assertion that guards it at simulation time.

Req ID	Requirement	Feature	Test ID	Assertion ID
RQ-M01	Zero operand returns the other operand	Zero-operand handling	D1	A_EARLY_EXIT, SB
RQ-M02	Equal operands return that value	Equal-operand handling	D2	A_EARLY_EXIT, SB
RQ-M03	GCD result is mathematically correct for general operand pairs	General computation	D3, D4, D5	SB
RQ-F01	FSM stays within legal state set	Legal-state invariant	D1–D8	A_STATE_LEGAL
RQ-F02	Early-exit takes IDLE to DONE in one cycle	Early-exit transition	D1, D2	A_EARLY_EXIT
RQ-F03	Non-trivial input drives IDLE to RUN, and RUN to DONE on convergence	General-path transition	D3, D4, D5	A_GENERAL_RUN, A_RUN_CONVERGE
RQ-F04	Output handshake returns to IDLE	Result acceptance	D1–D8	A_DONE_IDLE
RQ-H01	in_ready high only in IDLE	Input handshake gating	D1–D8	A_INRDY_IDLE
RQ-H02	out_valid high for entire DONE	Output handshake gating	D1–D8	A_OUTVLD_DONE, A_OUTVLD_HELD
RQ-H03	gcd_out stable under backpressure	Output stability	D7	A_OUT_STABLE

Req ID	Requirement	Feature	Test ID	Assertion ID
RQ-H04	`gcd_out` sourced from dedicated result register	Dedicated output register	D7	`A_RESULT_REG`
RQ-R01	Reset clears all registers and outputs synchronously	Reset values	D8	`A_RST_CLEAR`
RQ-R02	Module accepts input on first edge after reset release	Post-reset readiness	D8	`A_RST_READY`
RQ-L01	Early-exit latency is 1 cycle	Early-exit latency	D1, D2	`A_EARLY_EXIT`, SB
RQ-L02	General latency is $1 + N$ cycles	General-case latency	D3, D4	SB
RQ-T01	Back-to-back transactions without idle gap	Sequential throughput	D6	`A_DONE_IDLE`, SB
RQ-P01	Operands and `in_valid` held stable while `in_valid` high and `in_ready` low	Input handshake stability	D1–D8	`A_IN_STABLE`, `A_INVLD_HELD`

Req ID legend: M = Math F = FSM H = Handshake R = Reset L = Latency T = Throughput P = Protocol
Test ID legend — each Dn is a `gcd_test_directed` scenario, runnable on its own with `+SCENARIO=<name>`: D1 = zero D2 = equal D3 = max D4 = coprime D5 = pow2 D6 = back_to_back D7 = backpressure D8 = reset.
The smoke test (`gcd_test_smoke`) re-runs D1 and D6 for bring-up; the random test (`gcd_test_random`) re-exercises every requirement under constrained-random stimulus with random back-pressure. SB = checked by the scoreboard reference model rather than a dedicated assertion.

3 Verification Strategy

3.1 Approach

The test stratagy follows the standart stimulus-response model. Stimulus gets driven into the DUT, the DUT responses are observed and compared against the specifications excpected behaior via a reference model. The DUT is treated as a black box by the scoreboard, meaning only signals from the Top-level signal interface (see section 1) are observed. Assertions do have acesses to the internal state of the machine to allow for testing of behavior of the internal state of the FSM to observe if it complies with the mandated specifications.
Every requirement listed in the tracability Matrix in section 2 is hit by at least one test, observed by at least one assertion or scoreboard check, and closed against a coverage target in Section 5.

3.2 Stimulus Generation

Stimulus genration is split in 3 groups:

Smoke test: The smoke test is the bring-up test. It drives one or two valid transactions through the DUT with no back-pressure and no reset events. Its only job is to confirm the TB infrastructure works end-to-end before the deeper tests get a chance to expose DUT bugs.

Directed Test: For every feature row in the traceability matrix that is exercisable by stimulus, the directed test runs a dedicated sub-sequence targeting that feature. Each feature is hit deterministically and repeatably, with at least three distinct stimulus variants to avoid passing by coincidence on a single magic number. The directed test accepts a `+SCENARIO=<name>` plusarg (default `all`) which selects a single sub-sequence to run on its own — useful for isolating one feature without running the whole bucket.

Constrained random: A single constrained random sequence drives the DUT to test for edge cses. The constrains are tuned to hit coverage goals defined in section 5.

3.3 Result Checking

Three checking mechanisms run simultaneously:

1. **Reference model.** The DUT processes one operand pair at a time, so the scoreboard tracks a single outstanding transaction. On each captured `(a, b)` it computes the golden value via `gcd_ref(a, b)` (modulo Euclid) and the expected latency, records the input timestamp, and sets a `pending` flag. On the following `out_valid` rising edge it compares the observed `gcd_out` against the golden value and the measured latency (derived from `$time`) against the expected latency, then clears `pending`. On reset it clears `pending` and the in-flight expectation.

2. **Assertions.** SystemVerilog assertions check the spec's protocol and timing rules every clock cycle. Every assertion is silenced while reset is active, matching the synchronous-reset behaviour. Assertions mostly check external pins, with a small number of exceptions that observe the internal FSM to confirm it follows the specified behaviour.

3. **Pending-flag protocol check.** The same `pending` flag flags unpaired traffic: a new input arriving while a result is still outstanding, or a result appearing with no pending input, each raises a `UVM_ERROR`. The scoreboard's `report_phase` prints the total/failed check tally at the end of the test.

3.4 Reusability and Runtime Control

The TB is built to be reusable and configurable without recompiling. Per-test behaviour is controlled at run time via plusargs and `uvm_config_db`:

- `+UVM_TESTNAME=<class>` picks the test.
- `-sv_seed <N>` seeds the randomisation engine.
- `+NUM_TX=<N>` sets the constrained-random transaction count.
- `+SCENARIO=<name>` selects a single directed scenario (operand class, back-to-back, back-pressure, or reset).
- `+UVM_VERBOSITY=<level>` tunes logging.
- `+UVM_TIMEOUT=<time>,YES` bounds runaway sims.

`WIDTH` is set at compile time via `+define+TB_WIDTH=N` (see Section 1).

The same compiled image then runs over the cross-product of (test × seed × back-pressure × tx count × scenario).

4 Testbench Architecture

The testbench is a single-file UVM environment. The DUT exposes two independent ready/valid channels plus a reset, so the testbench mirrors the real chip with **three active agents** — one per interface — coordinated by a virtual sequencer. Figure 1 shows the structure.

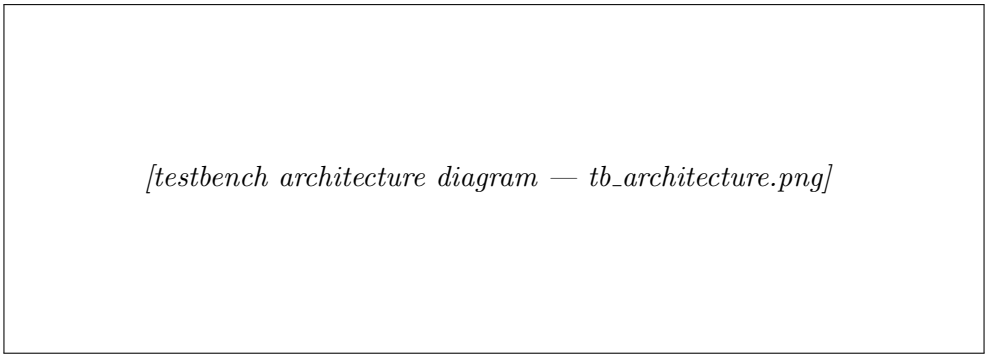

Figure 1: Testbench architecture. Solid arrows: pin-level signals. Dashed: TLM analysis ports. Dotted: `bind`.

Interface

A single `gcd_if` carries all DUT signals. `rst_n` is interface-owned (driven by the reset agent, not the top module). Each user gets its own clocking block and modport: `cb_in`/`mp_in` (drives `in_valid`/`a_in`/`b_in`), `cb_out`/`mp_out` (drives `out_ready`), `cb_mon`/`mp_mon` (samples everything, drives nothing), and `cb_rst`/`mp_rst` (drives `rst_n`). All driving and sampling goes through the clocking blocks, eliminating TB/DUT races.

Sequence items

- `gcd_in_tx` — input stimulus: `a_in`, `b_in`, `in_delay`.
- `gcd_out_tx` — output back-pressure stimulus: `ready`, `hold_cycles` (a segment of the `out_ready` waveform, independent of `out_valid`).
- `gcd_result_tx` — what the output monitor publishes: the captured `gcd_out`.
- `gcd_rst_tx` — reset stimulus: `assert_cycles`.

Handshake bits (`in_valid`, `out_ready` level) are *not* transaction fields — the drivers own the protocol mechanics.

Agents

Input agent (active): sequencer + driver + monitor. The driver runs the `in_valid`/`in_ready` handshake; the monitor publishes a `gcd_in_tx` on each captured input handshake.

Output agent (active): drives `out_ready` as a stream of waveform segments; the monitor publishes a `gcd_result_tx` on `ap_avail` at the `out_valid` rising edge (result available, used for the value and latency checks).

Reset agent (active): drives `rst_n` (power-on reset plus any mid-flight pulses); the reset monitor publishes a pulse on each `rst_n` falling edge.

Virtual sequencer and sequences

The virtual sequencer holds handles to the three channel sequencers. Virtual sequences coordinate the channels by starting sub-sequences on those handles: the directed virtual sequence pairs one directed input scenario with the matching output behaviour (and, for the reset scenario, a mid-flight reset pulse); the random virtual sequence forks a random input stream against a random back-pressure stream.

Scoreboard

The scoreboard receives the input, result-available, and reset streams. On each input it computes the golden value via `gcd_ref(a,b)` (modulo Euclid — an algorithm independent of the DUT's subtractive implementation) and the expected latency via a subtractive step count that mirrors the RTL. On the result-available event it checks the observed `gcd_out` against the golden value and the measured latency (from `$time`) against the expected latency. On reset it wipes the in-flight state. A pending flag detects unpaired inputs/outputs; a `report_phase` prints the pass/fail tally.

Coverage

A subscriber samples a covergroup every clock (handshake-signal crosses, operand and result value classes, the operand cross, equal-operand, and the FSM state with its legal transitions) plus a back-pressure stall-duration covergroup sampled at each output handshake and a reset covergroup sampled on each reset assertion. Details in Section 5.

Assertions

A separate `gcd_assertions` module is `bind`-ed into the DUT, giving it white-box access to `state`, `result_reg`, and the next-cycle datapath values without modifying the synthesizable RTL.

Top

`top` generates the clock, instantiates the interface (clock only) and the DUT, publishes the per-modport virtual interfaces via `uvm_config_db`, binds the assertions, and calls `run_test()`.

5 Coverage Goals

Closure uses both functional and code coverage.

Functional coverage

Sampled by the coverage subscriber. Goal: every bin hit at least once.

Coverpoint / cross	Bins / intent
handshake state (in)	cross `in_valid`×`in_ready`: handshake complete, input-not-valid, gcd-not-ready, idle
handshake state (out)	cross `out_ready`×`out_valid`: handshake complete, result-waiting, taker-waiting, idle
`a_in` / `b_in`	operand value class: zero, one, low, mid, high, max
`a`×`b` cross	both-low, both-high, a-low/b-high, a-high/b-low, a-zero, b-zero, both-zero
`a_eq_b`	equal vs not-equal operands (RQ-M02)
`gcd_out` class	result value class: zero, one, low, mid, high, max
`fsm_state`	FSM state derived from `in_ready`/`out_valid`: IDLE, RUN, DONE, plus transitions IDLE→RUN, IDLE→DONE, RUN→DONE, DONE→IDLE (RQ-F01–F04)
back-pressure stall	cycles `out_valid` is held before the output handshake: none, 1, 2–3, ≥ 4 (RQ-H03)
reset state	reset asserted during IDLE / RUN / DONE (RQ-R01/R02; RUN & DONE prove mid-flight reset)

Code coverage

Collected with `vlog +cover=bcefst` (branch, condition, expression, FSM, statement, toggle) and merged across tests with `vcover merge`. Goal: $\geq 95\%$ statement/branch/FSM, with any gap explained (e.g. unreachable `default` states).

Per-test progression

A separate coverage database (`.ucdb`) is saved per test (smoke, directed, random) so coverage growth over the test sequence is visible, then merged for the final closure figure.

6 Closure Criteria

Verification is considered closed when all of the following hold:

1. **All tests pass** — smoke, directed (all scenarios), and random complete with zero `UVM_ERROR` and zero `UVM_FATAL`.
2. **Scoreboard clean** — the `report_phase` reports zero failed value or latency checks.
3. **No assertion failures** — every bound SVA property holds across all runs.
4. **Functional coverage** — 100% of the functional bins in Section 5 are hit (including the mid-flight reset and equal-operand bins, which need the directed reset / random stimulus to close).
5. **Code coverage** — merged code coverage meets the Section 5 target, with any residual gap justified.

6. **Injected-bug check** — the deliberately injected RTL bug is caught by at least one test or assertion, demonstrating the testbench's detection ability.